

Relazione Progetto DevSecOps su AWS

Luciano Zappulla

2 febbraio 2026

Sommario

Questo documento descrive l'architettura e l'implementazione di una pipeline DevSecOps completa su Amazon Web Services (AWS). Il progetto dimostra il deployment sicuro di un'applicazione Python Flask containerizzata su Amazon ECS Fargate, integrando controlli di sicurezza automatizzati (SAST, SCA, IaC Scanning) e gestione delle notifiche tramite SNS e SQS.

Indice

1	Introduzione	2
2	Architettura del Sistema	2
2.1	Networking (VPC)	2
2.2	Compute (Amazon ECS Fargate)	2
2.3	Application Load Balancer (ALB)	2
2.4	Autenticazione (Amazon Cognito)	2
3	Pipeline DevSecOps (CI/CD)	2
3.1	Sorgente	2
3.2	Build e Sicurezza (AWS CodeBuild)	3
3.3	Deployment	3
4	Notifiche e Monitoraggio	3
5	Infrastructure as Code (IaC)	3
6	Conclusione	4

1 Introduzione

L'obiettivo del progetto è realizzare un ciclo di rilascio software sicuro e automatizzato, seguendo i paradigmi DevSecOps. L'infrastruttura è definita interamente tramite codice (Infrastructure as Code) utilizzando AWS CloudFormation, garantendo riproducibilità e versionamento.

2 Architettura del Sistema

L'architettura è progettata per essere scalabile, resiliente e sicura. I componenti principali includono:

2.1 Networking (VPC)

È stata creata una Virtual Private Cloud (VPC) con:

- Subnet Pubbliche: Per il Load Balancer e il NAT Gateway.
- Subnet Private: Per i workload ECS, garantendo che non siano direttamente esposti a Internet.
- Security Groups: Per il controllo granulare del traffico di rete.

2.2 Compute (Amazon ECS Fargate)

L'applicazione è ospitata su Amazon Elastic Container Service (ECS) utilizzando il launch type Fargate, che elimina la necessità di gestire i server sottostanti (Serverless compute for containers).

2.3 Application Load Balancer (ALB)

Un ALB gestisce il traffico in entrata e lo distribuisce ai task ECS. È configurato per integrarsi con Amazon Cognito per l'autenticazione degli utenti prima di inoltrare le richieste all'applicazione.

2.4 Autenticazione (Amazon Cognito)

Amazon Cognito User Pool è utilizzato per gestire le identità degli utenti. L'ALB verifica la sessione dell'utente tramite Cognito, garantendo che solo gli utenti autenticati possano accedere agli endpoint protetti.

3 Pipeline DevSecOps (CI/CD)

La pipeline di Continuous Integration e Continuous Deployment è orchestrata da AWS CodePipeline e AWS CodeBuild.

3.1 Sorgente

Il codice risiede su GitHub. Ogni push sul branch `main` innesca l'esecuzione della pipeline.

3.2 Build e Sicurezza (AWS CodeBuild)

Questa fase esegue i seguenti step definiti in `buildspec.yml`:

1. **Unit Testing**: Esecuzione dei test funzionali con **pytest**.
2. **SAST (Static Application Security Testing)**: Scansione del codice sorgente con **Semgrep** (configurazione OWASP Top 10) per individuare vulnerabilità nel codice Python.
3. **IaC Scanning**: Scansione dei template CloudFormation con **Checkov** per rilevare misconfigurazioni infrastrutturali.
4. **Container Build**: Costruzione dell'immagine Docker.
5. **Container Scanning**: Analisi dell'immagine Docker con **Trivy** per individuare vulnerabilità (CVE) nelle librerie di sistema e dipendenze.
6. **Push**: Caricamento dell'immagine sicura su Amazon Elastic Container Registry (ECR).

3.3 Deployment

CodePipeline aggiorna il servizio ECS con la nuova definizione del task, avviando il rolling update dei contenitori.

4 Notifiche e Monitoraggio

Per garantire visibilità sugli eventi della pipeline, è stato implementato un sistema di notifiche basato su:

- **Amazon SNS (Simple Notification Service)**: Un topic dedicato (`devsecops-notifications`) riceve eventi sullo stato della pipeline (Inizio, Successo, Fallimento).
- **CodeStar Notifications**: Una regola di notifica collega CodePipeline al topic SNS.
- **Amazon SQS (Simple Queue Service)**: Una coda (`devsecops-pipeline-events`) è iscritta al topic SNS per disaccoppiare la gestione degli eventi e permettere future elaborazioni asincrone o auditing.

5 Infrastructure as Code (IaC)

L'intera infrastruttura è modulare e definita nei seguenti template CloudFormation:

- `00-parameters.yml`: Parametri condivisi (es. nome progetto).
- `01-network.yml`: VPC, Subnet, Routing.
- `02-cognito.yml`: User Pool e Client App.
- `03-ecs.yml`: Cluster ECS, Task Definition, Service, ALB.
- `04-pipeline.yml`: CodePipeline, CodeBuild, Repository ECR e Regole di Notifica.
- `05-storage-notifications.yml`: Bucket S3 per gli artefatti, Topic SNS e Coda SQS.

6 Conclusione

Il progetto realizza un workflow moderno dove la sicurezza è integrata ("Shift Left") nel processo di sviluppo. L'uso di strumenti come Semgrep, Checkov e Trivy all'interno della pipeline assicura che solo codice e infrastruttura validati vengano rilasciati in produzione.